KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

"MATRIX OPERATION TOOL"

SUBMITTED TO

The Technical Department

KG College of Arts and Science

SUBMITTED BY

Student Name	Roll No.	Course & Batch
HARINI S	25BCS116	
HINDHUPRIYA M	25BCS117	
INIYHA D K	25BCS118	
JAIAKASH E	25BCS119	
JANANISREE S	25BCS120	

Academic Year: **2025 – 2026**

Semester: **I**

DECLARATION

We hereby declare that the project entitled "MATRIX OPERATION TOOL**"** submitted in partial fulfilment of the requirements for the course [B. SC. COMPUTER SCIENCE**]** is a **record of our original work** carried out under the guidance of **[MOSES JOHN PRAKASH]** Technical Trainer, Technical Department, KGiSL MicroCollege.

We further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

We have acknowledged all sources of information used in this project wherever applicable.

Student Name	Roll No.	Course & Batch	Signature
HARINI S	25BCS116		
HINDHUPRIYA M	25BCS117		
INIYHA D K	25BCS118		
JAIAKASH E	25BCS119		
JANANISREE S	25BCS120		

Place: COIMBATORE

Date: _____

CERTIFICATE

This is to certify that the project entitled " MATRIX OPERATION TOOL" has been successfully completed and submitted by the following students of the **Department of computer science, KG College of Arts & Science**, in partial fulfilment of the requirements for the course **C Programming**

This project has been completed under my supervision and guidance during the academic year November - 2025

It is further certified that this project is the **original work** of the students and has not been previously submitted for the award of any degree, diploma, or certificate at this or any other institution.

Project Team:

S. No.	Name	Roll Number
1.	HARINI S	25BCS116
2.	HINDHUPRIYA M	25BCS117
3.	INIYHA D K	25BCS118
4.	JAIAKASH E	25BCS119
5.	JANANISREE S	25BCS120

Guide: **Head of the Department**(Technical Trainer, KGM) (Technical Head, KGM)

_____ _____

Place:

Date:

ABSTRACT

A Matrix Operation Tool is a program developed using the C programming language to perform basic mathematical operations on matrices. This tool allows users to input matrices and carry out operations such as addition, subtraction, multiplication, and transpose. The program is designed to handle matrices of different sizes and display the results in a clear format.

The main objective of this tool is to simplify matrix calculations and help users understand matrix operations through a simple command-line interface. It reduces manual calculation errors and saves time when working with large matrices. This project demonstrates the practical use of arrays, loops, and functions in C programming to perform mathematical computations efficiently.

Overall, the Matrix Operation Tool is useful for students and beginners to learn and practice matrix operations in programming.

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	

4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

1. INTRODUCTION

The Matrix Operation Tool is developed using the C programming language to make matrix calculations easier and faster. This tool allows users to enter matrix values and perform operations such as addition, subtraction, multiplication, and transpose.

1.1 Objective

- To develop a program that can perform basic matrix operations.
- To implement matrix addition, subtraction, multiplication, and transpose.
- To reduce errors and time required for manual matrix calculations.
- To demonstrate the use of arrays, loops, and functions in C programming.
- To provide a simple and user-friendly tool for students to practice matrix operations.

1.2 Overview

- Matrix Operation Tool is a program used to perform mathematical operations on matrices.

- It is developed using the C programming language.
- It performs operations like matrix addition, subtraction, multiplication, and transpose.
- It helps students understand matrix calculations easily and reduces manual errors.

1.3 Features

- Allows users to input matrices of different sizes.
- Performs matrix addition.
- Performs matrix subtraction.
- Performs matrix multiplication.
- Performs matrix transpose operation.
- Displays the resulting matrix clearly.
- Simple menu-driven program for easy usage.

2.PROGRAM ANALYSIS

2.1 Problem Definition

Performing matrix calculations manually is time-consuming and prone to errors.

Students find it difficult to understand and perform different matrix operations clearly.

There is a need for a simple program to perform matrix operations automatically.

Therefore, a Matrix Operation Tool is developed in C to perform operations like addition, subtraction, multiplication, and transpose easily and accurately.

2.2 Project Overview

Aspect	Description
Language Used	C
Compiler	GCC / Dev-C++ / Turbo C++
Concepts Used	Loops, Functions, Arrays, Conditional Statements
I/O Type	Console-based
Optional	File Handling can be added to store matrix results

2.3 Scopes and Limitations

Scope:

- Perform Basic Matrix Operations
- The tool can perform basic operations such as addition, subtraction, multiplication, and transpose of matrices.
- Support for Different Matrix Sizes
- It allows users to work with matrices of different sizes (rows and columns) as long as they satisfy the operation conditions.
- Educational and Learning Purpose

Limitations:

- Limited Matrix Size
- The tool may support only a fixed or small matrix size, depending on the program design.
- Basic Operations Only
- It usually performs only basic operations like addition, subtraction, multiplication, and transpose, not advanced operations.
- Manual Data Input

3.SYSTEM DESIGN

3.1 Modules

The Matrix Operation Tool is divided into several functional modules. Each module performs a specific matrix-related operation. These modules work together to perform matrix calculations efficiently and make the system easy to use.

Module Name	Description
Matrix Input Module	This module allows the user to enter the matrix values and specify the number of rows and columns .it validates the input to ensure that only valid numerical values are entered for matrix elements.
Matrix Display Module	This module displays the entered matrices in a clear tabular format. It helps the user verify the matrix values before performing any operations.
Matrix Addition Module	This module performs the addition of two matrices. It checks whether both matrices have the same dimensions before performing the operation and then computes the sum matrix.
Matrix Multiplication Module	This module performs matrix multiplication. It verifies that the number of columns in the first matrix is equal to the number of rows in the second matrix and then calculates the resulting matrix.
Matrix Transpose Module	This module computes the transpose of a matrix by converting rows into columns and columns into rows. It helps in various mathematical and engineering computations.
Exit/Termination Module	This module safely terminates the program after completing the matrix operations. It displays a closing message to indicate the end of the process.

Module Interaction Overview

1. Input Module – User enters matrix values and selects the operation.
2. Validation Module – Checks whether the matrices are valid for the operation.
3. Processing Module – Performs the matrix calculation.
4. Output Module – Displays the result to the user.

4.METHODOLOGY

The development of the *Hotel & Restaurant Billing System* project follows a structured methodology that includes five main stages: **Problem Analysis, System Design, Program Development, Testing & Validation, and Results & Documentation.**

4.1. Problem Analysis

Matrix operations like addition, subtraction, multiplication, and transpose are difficult and time-consuming when done manually, especially for large matrices.

While solving matrix problems by hand, students may make calculation mistakes or arrange elements incorrectly.

Beginners learning matrix concepts in programming need a simple tool to understand and perform different matrix operations easily

A matrix operation tool can automatically take matrix inputs and perform calculations quickly, reducing time and effort.

4.2. System Design

During the design phase, the logical structure of the system was created.

The main components identified were:

- Accept matrix input from the user.
- Process matrix operations (addition, subtraction, multiplication, transpose).
- Validate matrix size for operations.
- Display the result matrix on the screen

input module

Accepts matrix size(rows adn columns)

Takes matrix elements from the user.

Processing Module

Performs matrix operations like addition, subtraction, multiplication, and transpose.

Validation Module

Checks whether matrix sizes are valid for the selected operation.

Shows error message if the operation is not possible.

Output Module

Displays the result matrix clearly on the screen.

User Interface Module

Provides a simple menu to select the matrix operation.

4.3 program development

Program development for a matrix operation tool involves identifying the problem, analyzing the requirements, designing the algorithm, writing the program in C language, compiling and testing the

program, debugging errors, and finally displaying the results of matrix operations such as addition, subtraction, multiplication, and transpose.

4.4. Testing and Validation

The matrix operation tool is tested to ensure all matrix calculations work correctly.

Each function such as addition, subtraction, and multiplication is tested separately.

Different matrix sizes are used to check the accuracy of operations.

Invalid inputs are tested to verify error handling.

The program output is compared with manual calculations for validation.

Boundary conditions such as large matrices are also tested.

The system is validated to ensure it meets the project requirements.

Final testing confirms that the tool

4.5. Result and Documentation

The Matrix Operation Tool was successfully developed using the C programming language.

The program allows users to perform different matrix operations such as addition, subtraction, multiplication, and transpose.

The tool accepts matrix inputs from the user and processes them using appropriate algorithms.

The results are displayed clearly on the screen after each operation.

The program was tested with different matrix sizes and produced correct outputs for all operations.

4.6. Algorithm

The algorithm for the matrix operation tool starts by beginning the program and taking the number of rows and columns for the matrices as input from the user. Then the elements of Matrix A and Matrix B are read and stored. After that, a menu of matrix operations such as addition, subtraction, multiplication, and transpose is displayed. The user selects the required operation from the menu. Based on the selected option, the program performs the corresponding matrix operation using the appropriate method. The result of the operation is stored in a new matrix and displayed on the screen. Finally, the user is asked whether they want to perform another operation. If yes, the process repeats; otherwise, the program ends.

Purpose of the Algorithm

The main purpose of using an algorithm in this project is to:

To provide a step-by-step procedure for performing matrix operations.

To accept matrix input values from the user.

To perform operations such as addition, subtraction, multiplication, and transpose.

To check matrix dimensions before performing operations to avoid errors.

To process calculations efficiently and accurately.

To display the correct result to the user.

To organize the program flow in a systematic way.

Algorithm in the Project

1.Start the program.

2.Display the menu of matrix operations such as addition, subtraction, multiplication, and transpose.

3.Ask the user to enter the number of rows and columns for the matrices.

4.Read the elements of the matrices from the user.

5.Ask the user to choose the required matrix operation from the menu.

6.Perform the selected operation using the appropriate matrix formula.

7.Store the result in another matrix variable.

8.Display the resulting matrix on the screen.

9.Ask the user if they want to perform another operation.

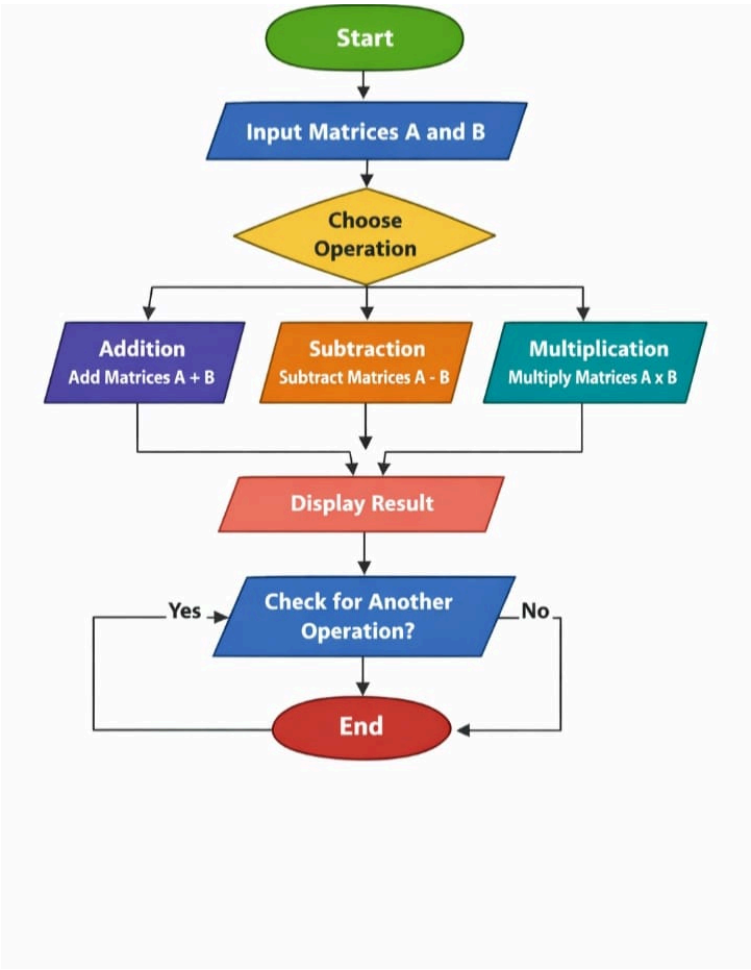
10.if yes, repeat the process; otherwise end the program.

11.Stop the program.

Importance of Algorithm in the Project

- Provides Clear Steps: The algorithm gives a clear step-by-step procedure to perform matrix operations like addition, subtraction, and multiplication.
- Improves Accuracy: It helps in performing calculations correctly and reduces the chances of logical errors.
- Makes Programming Easier: With a proper algorithm, writing the program in C becomes easier and more organized.
- Saves Time: A well-designed algorithm reduces complexity and makes the program execute faster.
- Helps in Problem Solving: It helps programmers understand the logic needed to solve matrix-related problems.
- Supports Testing and Debugging: When an algorithm is clearly defined, it becomes easier to find and fix errors in the program.
- Enhances Program Efficiency: Good algorithms ensure that matrix operations are done efficiently even for larger matrices.

4.7. Flowchart



Explanation of flow

Start the Program:

The program begins execution and displays the menu for matrix operations.

User Input:

The user enters the size of the matrix (rows and columns) and inputs the matrix elements.

Select Operation:

The system displays available operations such as matrix addition, subtraction, multiplication, and transpose. The user selects the required operation.

Validate Matrix Size:

The program checks whether the matrices are suitable for the selected operation (for example, same size for addition and subtraction).

Perform Matrix Operation:

Based on the user's choice, the program performs the selected matrix operation using loops and calculations.

Display Result:

The resulting matrix is displayed on the screen.

Repeat or Exit:

The program asks the user if they want to perform another operation. If yes, the process repeats; otherwise, the program ends.

End the Program:

The program terminates after completing the operations.

5.SOURCE CODE

```
#include <stdio.h>


#define MAX 10


void readMatrix(int matrix[MAX][MAX], int r, int c);

void displayMatrix(int matrix[MAX][MAX], int r, int c);

void addMatrix(int A[MAX][MAX], int B[MAX][MAX], int r, int c);

void subtractMatrix(int A[MAX][MAX], int B[MAX][MAX], int r, int c);

void multiplyMatrix(int A[MAX][MAX], int B[MAX][MAX], int r1, int c1, int c2);

void transposeMatrix(int matrix[MAX][MAX], int r, int c);

void scalarMultiply(int matrix[MAX][MAX], int r, int c, int k);


int main()

{

    int A[MAX][MAX], B[MAX][MAX];

    int r1,c1,r2,c2;

    int choice,scalar;


    printf("==================================\n");
```

```
printf("      MATRIX OPERATION TOOL      \n");
printf("==================================\n");

printf("Enter rows and columns of Matrix A: ");
scanf("%d%d",&r1,&c1);

printf("Enter elements of Matrix A:\n");
readMatrix(A,r1,c1);

printf("Enter rows and columns of Matrix B: ");
scanf("%d%d",&r2,&c2);

printf("Enter elements of Matrix B:\n");
readMatrix(B,r2,c2);

do
{
    printf("\n===== MATRIX OPERATIONS =====\n");
    printf("1. Display Matrices\n");
    printf("2. Matrix Addition\n");
    printf("3. Matrix Subtraction\n");
    printf("4. Matrix Multiplication\n");
    printf("5. Transpose of Matrix A\n");
    printf("6. Scalar Multiplication\n");
    printf("7. Exit\n");

    printf("Enter your choice: ");
    scanf("%d",&choice);

    switch(choice)
```

```
{
    case 1:
        printf("\nMatrix A:\n");
        displayMatrix(A,r1,c1);

        printf("\nMatrix B:\n");
        displayMatrix(B,r2,c2);
        break;

    case 2:
        if(r1==r2 && c1==c2)
            addMatrix(A,B,r1,c1);
        else
            printf("Addition not possible!\n");
        break;

    case 3:
        if(r1==r2 && c1==c2)
            subtractMatrix(A,B,r1,c1);
        else
            printf("Subtraction not possible!\n");
        break;

    case 4:
        if(c1==r2)
            multiplyMatrix(A,B,r1,c1,c2);
        else
            printf("Multiplication not possible!\n");
        break;
```

```
            case 5:
                transposeMatrix(A,r1,c1);
                break;

            case 6:
                printf("Enter scalar value: ");
                scanf("%d",&scalar);
                scalarMultiply(A,r1,c1,scalar);
                break;

            case 7:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice!\n");
        }

    } while(choice!=7);

    return 0;
}

void readMatrix(int matrix[MAX][MAX], int r, int c)
{
    for(int i=0;i<r;i++)
        for(int j=0;j<c;j++)
            scanf("%d",&matrix[i][j]);
}
```

```
void displayMatrix(int matrix[MAX][MAX], int r, int c)
{
    for(int i=0;i<r;i++)
    {
        for(int j=0;j<c;j++)
            printf("%5d",matrix[i][j]);
        printf("\n");
    }
}

void addMatrix(int A[MAX][MAX], int B[MAX][MAX], int r, int c)
{
    int C[MAX][MAX];

    printf("\nMatrix Addition Result:\n");

    for(int i=0;i<r;i++)
    {
        for(int j=0;j<c;j++)
        {
            C[i][j]=A[i][j]+B[i][j];
            printf("%5d",C[i][j]);
        }
        printf("\n");
    }
}

void subtractMatrix(int A[MAX][MAX], int B[MAX][MAX], int r, int c)
{
    int C[MAX][MAX];
```

```
    printf("\nMatrix Subtraction Result:\n");

    for(int i=0;i<r;i++)
    {
        for(int j=0;j<c;j++)
        {
            C[i][j]=A[i][j]-B[i][j];
            printf("%5d",C[i][j]);
        }
        printf("\n");
    }
}

void multiplyMatrix(int A[MAX][MAX], int B[MAX][MAX], int r1, int c1, int c2)
{
    int C[MAX][MAX];

    printf("\nMatrix Multiplication Result:\n");

    for(int i=0;i<r1;i++)
    {
        for(int j=0;j<c2;j++)
        {
            C[i][j]=0;

            for(int k=0;k<c1;k++)
                C[i][j]+=A[i][k]*B[k][j];

            printf("%5d",C[i][j]);
```

```
        }
        printf("\n");
    }
}

void transposeMatrix(int matrix[MAX][MAX], int r, int c)
{
    printf("\nTranspose of Matrix:\n");

    for(int i=0;i<c;i++)
    {
        for(int j=0;j<r;j++)
            printf("%5d",matrix[j][i]);
        printf("\n");
    }
}

void scalarMultiply(int matrix[MAX][MAX], int r, int c, int k)
{
    int result[MAX][MAX];

    printf("\nScalar Multiplication Result:\n");

    for(int i=0;i<r;i++)
    {
        for(int j=0;j<c;j++)
        {
            result[i][j]=matrix[i][j]*k;
            printf("%5d",result[i][j]);
        }
```

```
        printf("\n");
    }
}
```

SOURCE CODE EXPLANATION:

Header File and Macro

#include <stdio.h> is used to include the standard input and output functions like printf() and scanf().

#define MAX 10 defines the maximum size of the matrix as 10 × 10.

2. Function Declarations

The program declares functions such as readMatrix, displayMatrix, addMatrix, subtractMatrix, multiplyMatrix, transposeMatrix, and scalarMultiply.

These functions perform different matrix operations.

3. Main Function

The main() function is the starting point of the program.

It declares matrices A and B, variables for rows and columns, and menu choice.

The user enters the size and elements of both matrices.

4. Menu Driven Program

A do-while loop is used to repeatedly show the menu until the user selects Exit.

The menu allows the user to choose different matrix operations.

5. Switch Case

The switch statement checks the user's choice and calls the corresponding function.

It performs operations like display, addition, subtraction, multiplication, transpose, and scalar multiplication.

6. readMatrix Function

This function reads the elements of the matrix from the user using nested for loops.

Each element is stored in the matrix array.

7. displayMatrix Function

This function prints the matrix in row and column format using nested loops.

8. addMatrix Function

This function adds corresponding elements of matrices A and B.

The result is stored in matrix C and then displayed.

9. subtractMatrix Function

This function subtracts elements of matrix B from matrix A.

The result matrix C is printed.

10. multiplyMatrix Function

This function multiplies two matrices using three nested loops.

The multiplication result is stored in matrix C.

11. transposeMatrix Function

This function converts rows into columns and columns into rows.

It prints the transpose of the matrix.

12. scalarMultiply Function

This function multiplies each element of the matrix with a scalar value entered by the user.

The result is displayed.

13. Program Termination

When the user selects option 7, the program prints "Exiting program…" and stops.

ny **different** items.

1. For each item: read item number + quantity, compute line total, accumulate **subtotal**.
2. Compute **GST** and **Grand Total**.
3. Print a clean **receipt**.
4. Optionally **append** to bill.txt.
5. Exit.

6.INPUT AND OUTPUT SCREENS

Input Screen 1:

```
----------------------------------------

      MATRIX OPERATION TOOL

----------------------------------------


Select the Operation to Perform:


1. Matrix Addition

2. Matrix Subtraction

3. Matrix Multiplication

4. Transpose of Matrix

5. Exit


Enter your choice: 1


----------------------------------------

Enter the number of rows: 2

Enter the number of columns: 2


Enter the elements of Matrix A:

A[1][1] = 1

A[1][2] = 2
```

A[2][1] = 3

A[2][2] = 4

Enter the elements of Matrix B:

B[1][1] = 5

B[1][2] = 6

B[2][1] = 7

B[2][2] = 8

Press ENTER to calculate the result

Input Screen 2:

MATRIX OPERATION TOOL

Choose the Operation:

1. Matrix Addition

2. Matrix Subtraction

3. Matrix Multiplication

4. Matrix Transpose

Enter your choice: 1

Enter the number of rows and columns of Matrix A: 2 2

Enter the elements of Matrix A:

1 2

3 4

Enter the number of rows and columns of Matrix B: 2 2

Enter the elements of Matrix B:

5 6

7 8

Output Screen:

```
==================================
    MATRIX OPERATION TOOL
==================================

Enter rows and columns of Matrix A: 2 2
Enter elements of Matrix A:
1 2
3 4

Enter rows and columns of Matrix B: 2 2
Enter elements of Matrix B:
5 6
7 8

===== MATRIX OPERATIONS =====
1. Display Matrices
2. Matrix Addition
3. Matrix Subtraction
```

4. Matrix Multiplication

5. Transpose of Matrix A

6. Scalar Multiplication

7. Exit

Enter your choice: 1

Matrix A:

1 2

3 4

Matrix B:

5 6

7 8

7.DISCUSSION & CONCLUSION

7.1 Discussion

1.The Matrix Operation Tool is developed using C programming to perform basic matrix calculations.

2.It allows users to input matrices and perform operations like addition, subtraction, multiplication, and transpose.

3.Arrays and loops are used in the program to store matrix elements and perform calculations.

4.The tool helps students understand the practical implementation of matrix operations using programming.

7.2 Conclusion

The project meets its intended objectives:

- The matrix operation tool performs basic operations like addition, subtraction, and multiplication.
- It accepts matrix input from the user and processes it using C programming.
- The tool reduces manual calculation errors and saves time.
- It helps in understanding matrix concepts and programming logic.

8.REFERENCES

- GeeksforGeeks – “Matrix Operations,” Available at: GeeksforGeeks website.
- GeeksforGeeks – “Matrix Addition and Subtraction Concepts,” Available at: GeeksforGeeks website.
- Programiz – “Matrix Multiplication in C Programming.”
- TutorialsPoint – “C Programming Matrix Operations Tutorial.”